

Rapport de projet

Systeme de Gestion des Auditoires (SGA)

1. Introduction

Le projet SGA est une application web de gestion academique destinee a organiser l'occupation des auditorios et la planification hebdomadaire des cours.

Il repond a un besoin frequent dans les etablissements d'enseignement: affecter des cours a des salles et a des creneaux horaires en respectant des contraintes de capacite et de non-conflit.

2. Contexte et problematique

Dans une faculte, plusieurs promotions (L1 a L4) suivent des cours de tronc commun et d'option. La planification manuelle est longue, source d'erreurs et difficile a maintenir.

Le projet SGA propose une solution centralisee qui permet:

- de gerer les entites academiques (salles, promotions, options, cours),
- de generer automatiquement un planning,
- de controler la coherence du resultat,
- d'ajuster manuellement le planning si necessaire.

3. Objectifs du projet

Objectif general: automatiser et securiser la gestion du planning academique.

Objectifs specifiques:

- Mettre a disposition des interfaces CRUD pour les donnees de base.
- Generer un planning hebdomadaire sans conflits majeurs.
- Verifier automatiquement les conflits de salle, de promotion et de capacite.
- Proposer un rapport d'occupation des salles.
- Securiser l'acces a l'application (authentification + 2FA + CSRF).

4. Analyse du besoin fonctionnel

4.1 Acteurs

- Administrateur academique: gere les donnees, lance la generation, modifie le planning et consulte les rapports.

4.2 Besoins fonctionnels

- Authentification securisee avec double facteur (OTP).
- Gestion des salles (nom, capacite).
- Gestion des promotions (niveau, effectif).
- Gestion des options (L3/L4).
- Gestion des cours (type, niveau, option, duree).
- Generation automatique du planning.
- Consultation du planning (tableau + grille hebdomadaire).
- Modification manuelle des affectations avec validation.
- Rapport d'occupation globale et par salle.

4.3 Contraintes metier

- Une salle ne peut accueillir qu'un cours par creneau.
- Une promotion ne peut avoir deux cours simultanes.
- La capacite de la salle doit couvrir l'effectif de la promotion.
- 1 creneau = 4 heures.

- Journee type: 08:00-12:00 (cours 1), pause 12:00-13:00, 13:00-17:00 (cours 2).

5. Analyse non fonctionnelle

- Simplicite de deploiement: application PHP sous WAMP.
- Maintenablete: logique partagee centralisee dans includes/.
- Performance: adaptee a de petites et moyennes volumétries.
- Securite: protections de base (XSS, CSRF, hash mot de passe, OTP).
- Limite: persistance JSON (pas de concurrence transactionnelle robuste).

6. Architecture et structure

6.1 Organisation

- Pages applicatives: login.php, dashboard.php, salles.php, cours.php, planning_generate.php, planning_view.php, planning_edit.php, rapport.php.
- Noyau partage (includes/): functions.php, security.php, auth.php, planning.php.
- Donnees (data/): salles.json, promotions.json, options.json, cours.json, planning.json, users.json.
- Front: assets/css/style.css et assets/js/app.js.

6.2 Style architectural

Le projet suit une architecture monolithique legere et procedurale modulaire:

- traitement serveur PHP,
- separation simple logique/affichage,
- fonctions metier reutilisables via require_once.

7. Choix de conception

7.1 Persistance JSON

Avantages:

- simplicite de mise en place,
- lisibilite des donnees,
- deploiement rapide.

Inconvenients:

- scalabilite limitee,
- absence de garanties ACID fortes,
- gestion multi-utilisateur complexe.

7.2 Algorithme de planification

Le generateur utilise une approche gloutonne (greedy):

- tri des cours selon la capacite requise,
- recherche d'un creneau valide,
- affectation d'une salle compatible,
- marquage des ressources occupees.

Avantage: rapide et simple.

Limite: pas d'optimalite globale garantie.

7.3 Securite

- password_hash/password_verify.
- Token CSRF sur formulaires.
- Echappement HTML systematique.

- OTP avec expiration pour la 2FA.

8. Flux de fonctionnement

1. Connexion email/mot de passe.
2. Verification OTP.
3. Acces au tableau de bord.
4. Gestion des donnees de base.
5. Generation du planning.
6. Consultation et controle des conflits.
7. Ajustements manuels si besoin.
8. Consultation du rapport d'occupation.

9. Resultats et apports

Le systeme permet:

- centralisation des donnees academiques,
- reduction des erreurs de planification,
- meilleure visibilite sur l'utilisation des salles,
- gain de temps administratif.

10. Limites identifiees

- Pas de base de donnees relationnelle.
- Pas de gestion avancee de concurrence.
- Pas de moteur d'optimisation avance.
- Couverture de tests automatisee a renforcer.

11. Recommandations

- Migrer vers MySQL/PostgreSQL.
- Introduire une couche service et un MVC progressif.
- Ajouter des tests unitaires et d'integration.
- Renforcer la 2FA (tentatives max, journalisation).
- Ajouter export CSV/PDF natif du planning.

12. Conclusion

Le projet SGA atteint son objectif principal: automatiser la planification des cours en respectant les contraintes essentielles.

L'architecture est pertinente pour un cadre academique.

Pour une exploitation plus large, une evolution vers une architecture plus robuste est recommandee.